```
#define BLACK   0x0000
#define BLUE    0x001F
#define GREEN   0x07E0
#define WHITE   0xFFFF
#define GRAY    0x8410
#define ORANGE  0xFA60
#define LIME    0x07FF
#define AQUA    0x04FF
#define WALL    0x4E72
#define MAROON  0xC845
#define GREE    0x5DC0
```

Fig. 9: Setting colour code for display

Create a function to get the point of touch on the LCD screen in X and Y coordinates (Fig. 8).

Next, add codes for colours that we are going to use in the display (refer Fig. 9).

Then create a setup function where we will add codes to show buttons and all other elements on display (refer Figs. 10 and 11).

We need to create another function to check the state of buttons and assign their tasks. For example, if we touch a button, the assigned task changes the state of the relay to either on or off (refer Figs. 12 and 13).

We are done with the coding part. Save the code as touch_switch.ino. Compile it and upload the code into the Arduino Uno board. The next step is app development.

Final_switch_27_touch

```
void setup(void)
{
    Serial.begin(9600);
    pinMode(led,OUTPUT);
      pinMode(led2,OUTPUT);

    uint16_t ID = tft.readID();
    if (ID == 0xD3D3) ID = 0x9486; // write-only shield
    tft.begin(ID);
    tft.setRotation(0);            //PORTRAIT
    tft.fillScreen(BLACK);//left.up.size.size
    tft.setFont(&FreeMono9pt7b);
    on_btn.initButton(&tft,  60, 180, 100, 80, GREE, BLACK, GREE, "ON", 1);
    off_btn.initButton(&tft, 180, 180, 100, 80, AQUA, BLACK, AQUA, "OFF", 1);
    on_btn1.initButton(&tft,  60, 270, 100, 80, WALL, BLACK, WALL, "ON", 1);
    off_btn1.initButton(&tft, 180, 270, 100, 80, MAROON, BLACK, MAROON, "OFF", 1);
     on_btn.drawButton(false);
      off_btn.drawButton(false);
       on_btn1.drawButton(false);
        off_btn1.drawButton(false);
```

Fig. 10: Creating setup function

Android app development

App development is done on the MIT App Inventor. First, you need to have a valid Google email account. Next, open the link *https://appinventor.mit.edu* to start the MIT App Inventor project. You will find two tabs—Designer and Blocks on the top right side of the screen.

Open the Designer section, create a layout for the app, and then add the following components—four buttons, a list picker, and a Bluetooth client, as shown in Fig. 14.

Next, open the Blocks section in MIT App Inventor. Pick and place the code blocks, as shown in Fig. 15. Save the project as TouchSwitch.apk and install this .apk file on your Android phone.

Connections

Now, we need to connect the components and fit all the components and relay modules in a switch box or enclosure. The circuit diagram is shown in Fig. 16. The connection details between Arduino and other components are shown in Table 2.

```
tft.setCursor(2, 40);
tft.setTextColor(WHITE);
tft.setTextSize(1);
tft.print("SW 1");
tft.fillCircle(62, 35, 5, MAROON);
tft.drawCircle(62, 35, 7, WALL);
tft.setCursor(2, 56);
tft.print("SW 2");
tft.fillCircle(62, 54, 5, MAROON);
tft.drawCircle(62, 54, 7, WALL);
tft.setCursor(2, 73);
tft.print("BT S");
tft.fillCircle(62, 71, 5, MAROON);
tft.drawCircle(62, 71, 7, WALL);
tft.setCursor(2, 91);
tft.print("SW 4");
tft.fillCircle(62, 90, 5, MAROON);
tft.drawCircle(62, 90, 7, WALL);


}
```

Fig. 11: Creating display elements on the screen

```
}
Adafruit_GFX_Button *buttons[] = {&on_btn, &off_btn,&on_btn1, &off_btn1, NULL};
bool update_button(Adafruit_GFX_Button *b, bool down)
{
    b->press(down && b->contains(pixel_x, pixel_y));
    if (b->justReleased())
        b->drawButton(false);
    if (b->justPressed())
        b->drawButton(true);
    return down;
}
bool update_button_list(Adafruit_GFX_Button **pb)
{
    bool down = Touch_getXY();
    for (int i = 0 ; pb[i] != NULL; i++) {
        update_button(pb[i], down);
    }
    return down;
}

void loop(void)
{
if (Serial.available()){
  command=Serial.read();
  tft.fillCircle(62, 71, 5, GREE);

}
```

Fig. 12: Getting touch points